SOFTWARE ENGINEERING · LECTURE 2

Scan to follow along

Open today's slides on your own phone or laptop — then you can run real code yourself.

`hassanmiir.github.io/cpp-course/lecture2`



📱 Point your camera here

👁 0 opened today

Dr. Mir Hassan

Associate Professor in Artificial Intelligence and Trustworthy Digital Systems

LECTURE 2 · LET'S CONTINUE

Teaching the computer to remember things

Variables & Data Types — the building blocks of our University Portal project.

Store data | Numbers & text | Ask the user | Model a Student

A 2-hour hands-on session · building toward your project

QUICK RECAP

What we learned last time

`#include`	`main`	`cout`	`;`
Load a library	Where it starts	Print to screen	Ends a statement

The gap

Last time our program could only print *fixed* text. It couldn't remember a name, a score, or a number. Today we fix that.

WHY THIS MATTERS

Our project needs to remember things

Your **University Portal** stores a lot of information:

A Student
name, student number, year of study, GPA...

A Course
course code, title, credits, semester...

A Grade
marks, feedback, pass or fail...



Every one of these is **data** stored in **variables**.

OUR TWO PROJECTS

Both groups build the University Portal?

Same system, two focuses. Each group owns one side — then they connect.



Group A — Student & Courses

Registration & login, enrolling in courses, viewing your courses, attendance. Centred on the **Student** and **Enrollment**.



Group B — Teaching & Grades

Creating assignments, collecting submissions, grading & feedback, class reports. Centred on the **Lecturer** and **Grade**.

Today is shared

Both projects start from the same building block: storing a **Student** and a **Course** in variables. That's what we build now.

TODAY'S PLAN

What you'll be able to do



Variables

Create named boxes that hold data.



Data types

Pick int, double, char, bool, text.



Input

Read what the user types with cin.



Model a Student

Combine it all into a real record.

Look for this

The dark boxes with a green ▶ **Run** button are yours — type and run real C++, right in the slide.

PART 1

What is a variable?

A named box where your program keeps a piece of data.

- A box with a name and a value
- Declare it, then use it anywhere
- Your first variable — live!



THE BIG IDEA

A variable is a labelled box

Think of the computer's memory as a wall of boxes. A **variable** is one box that:

Has a name
So you can find it again — like `age` or `gpa`.

Holds a value
The data inside — like `20` or `3.75`.

Can change
That's why it's "variable" — the value can vary.



`age` → [20]

SYNTAX

How to make one

```
1 int age = 20;
```

`int`

The **type** — what kind of data (a whole number).

`age`

The **name** — how you refer to the box.

`= 20`

The **value** — what goes inside.

Read it as

"Make an integer box called age, and put 20 in it."

USING IT

Once it has a name, use it anywhere

```
1 int age = 20;
2 cout << "You are " << age << " years old." << endl;
3 age = 21;                          // change the value
4 cout << "Next year: " << age << endl;
```

Output

You are 20 years old.
Next year: 21

YOUR TURN · LIVE CODE RUN IT!

Your first variable

Run it. Then change the number, and add a second variable of your own.

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```cpp
#include <iostream>
using namespace std;

int main() {
    int studentNumber = 1024;
    cout << "Student number: " << studentNumber << endl;
    return 0;
}
```

OUTPUT Ready

Click ▶ Run to see the output here.

PART 2

The main data types

Different data needs different boxes.

- # int & double — numbers
- A char & text — letters and words
- bool — true or false



DATA TYPES

Five types you'll use all the time

int
Whole numbers: 20, -5, 1024

double
Decimals: 3.75, 99.9

char
One letter: 'A', 'B', 'F'

bool
true or false

string
Words & names: "Sara Khan"

Together
= one Student record

NUMBERS

`int` vs `double`

```
1 int credits = 6;         // whole number — no decimals
2 double gpa = 3.75;       // decimal number
3 int year = 2;
4 double tuition = 1499.99;
```

#

Use int for...

Counts: credits, year of study, number of students.

%

Use double for...

Measurements: GPA, average mark, money.

LETTERS & YES/NO

char ? and bool ?

```
1 char grade = 'A';          // ONE character, in single quotes
2 bool isEnrolled = true;    // only true or false
```



char

A single letter or symbol. Single quotes: `'A'`, not `"A"`.



bool

A yes/no switch. Perfect for "is the student enrolled?"

💡 Tip: click the underlined words ? above for a quick explanation.

YOUR TURN · LIVE CODE EXPERIMENT

One student, four types

Run it. Then change the values to describe *yourself* as a student.

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```cpp
#include <iostream>
using namespace std;

int main() {
    int    studentNumber = 1024;
    int    yearOfStudy   = 2;
    double gpa           = 3.75;
    char   grade         = 'A';
    bool   enrolled      = true;

    cout << "Number:   " << studentNumber << endl;
    cout << "Year:     " << yearOfStudy << endl;
```

OUTPUT Ready

Click ▶ Run to see the output here.

PART 3

Storing text

Names, titles, feedback — a Portal is full of text.

- A word in a char[] box
- std::string — the better way
- Store a student's name — live



TEXT

A word fits in a `char[]` box

```
1 char name[40] = "Sara";
2 cout << "Student: " << name << endl;
```

A `char[40]` is a box that holds up to 40 characters — enough for a name. The `[40]` is the maximum size.

> **Output**
> Student: Sara

THE BETTER WAY

Meet `std::string`

```cpp
#include <string>              // needed for string
using namespace std;

string name = "Sara Khan";     // holds a whole name, any length
string course = "Software Engineering";
```

Any length

No fixed `[40]` — it grows to fit the text.

Easy to use

Join, compare, and measure text simply. We'll go deep in Lecture 8.

In class

The quick in-slide runner uses `char[]`; in your full compiler (OnlineGDB) use `string` — that's what our project will use.

YOUR TURN · LIVE CODE RUN IT!

Store a student's name

Run it. Then change the name and add the student's `program` (e.g. "SoftwareEng").

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```
#include <iostream>
using namespace std;

int main() {
    char name[40]    = "Sara";
    int  studentNo   = 1024;
    double gpa       = 3.75;

    cout << "=== Student Card ===" << endl;
    cout << "Name:    " << name << endl;
    cout << "Number:  " << studentNo << endl;
    cout << "GPA:     " << gpa << endl;
```

OUTPUT Ready

Click ▶ Run to see the output here.

PART 4

Asking the user

A Portal is only useful if real people can enter their data.

- Read input with cin >>
- cout sends out, cin brings in
- A live registration form



INPUT

Read input with `cin`

```cpp
int age;
cout << "Enter your age: ";
cin >> age;                        // store what they type into age
cout << "You are " << age << endl;
```

`cout <<`

Arrows point OUT — send to the screen.

`cin >>`

Arrows point IN — read into a variable.

INPUT

Read several values at once

```
1 char name[40];
2 int  year;
3 cout << "Name and year of study: ";
4 cin >> name >> year;
5 cout << name << " is in year " << year << endl;
```

Chain `>>` to read several things in order. The user types them separated by spaces.

YOUR TURN · LIVE CODE INTERACTIVE

A mini registration form

The input box is pre-filled (Sara 2 3.75). Run it — then change the welcome message.

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```
#include <iostream>
using namespace std;

int main() {
    char name[40];
    int  year;
    double gpa;

    cout << "Enter name, year, GPA: ";
    cin >> name >> year >> gpa;

    cout << "--- Welcome to the Portal ---" << endl;
```

OUTPUT Ready

Click ▶ Run to see the output here.

PART 5

Rules & good habits

A few things that save you from bugs.

- Match the type to the data
- Name variables clearly
- const — values that never change



WATCH OUT

The whole-number trap

```
1 int total = 7;
2 int count = 2;
3 cout << total / count << endl;     // prints 3, not 3.5 !
```

Why?
Two `int`s divide as whole numbers — the `.5` is thrown away. For an average GPA, use `double`.

Fix: make at least one a `double` → `7.0 / 2` gives `3.5`.

GOOD HABITS

Clear names & `const`

Name for meaning

`gpa` and `yearOfStudy` beat `x` and `y`. Future-you will thank you.

`const` = never changes

`const int PASS_MARK = 50;` — a value fixed for the whole program.

In our project

Things like the pass mark or max credits are `const` — set once, used everywhere.

QUICK CHECK THINK!

Which type for a student's GPA?

A `int`

B `double`

C `char`

D `bool`

PREDICT THE OUTPUT THINK!

What prints?

```
int credits = 6;
credits = credits + 3;
cout << credits;
```

A 6

B 9

C 63

PART 6

Build it: a Student record

Put every type together — the first real piece of the Portal.

- Model one student with all types
- Compute something from the data
- Work with a partner



PAIR CHALLENGE · USE YOUR BRAIN 15 MIN

The Student Record Card

Build a program that stores a full student and prints a tidy card:

1. Store **name**, **student number**, **year**, and two test marks.
2. Think: compute the **average** of the two marks — as a `double`!
3. Trickier: a `bool passed` — did they average 50+?
4. Print it all inside a bordered card.

15:00

❚❚ Pause ↺ +1 min

The catch

Use `(m1 + m2) / 2.0` — the `.0` keeps the decimals. That's the whole-number trap from earlier!

PAIR CHALLENGE · BUILD HERE RUN IT!

Your Student Record Card

Starter stores the data. **You add** the average and the pass check. (Input pre-filled: Sara 1024 2 80 60)

main.cpp ▶ Run ↺ Open in OnlineGDB ▸

```
#include <iostream>
using namespace std;

int main() {
    char name[40];
    int  number, year, mark1, mark2;
    cin >> name >> number >> year >> mark1 >> mark2;

    // TODO 1: double average = (mark1 + mark2) / 2.0;
    // TODO 2: bool passed = average >= 50;   (we'll print it next lecture)

    cout << "==== STUDENT RECORD ====" << endl;
```

OUTPUT Ready

Click ▶ Run to see the output here.

ONE SOLUTION COMPARE YOURS

How the record works

```
double average = (mark1 + mark2) / 2.0;  // decimals kept
bool passed = average >= 50;

cout << "==== STUDENT RECORD ====" << endl;
cout << "Name:    " << name << endl;
cout << "Number:  " << number << endl;
cout << "Year:    " << year << endl;
cout << "Average: " << average << endl;
cout << "======================" << endl;
```

`/ 2.0` keeps decimals

80 and 60 average to 70.0, not 70 by luck.

`bool passed`

A comparison gives true/false — the seed of decisions (Lecture 4).

That's a real record

Group A's **Student** and Group B's gradebook row both start exactly here.

PART 7

Now: your two projects

You can store data. Let's turn that into the University Portal? you're building.



- Group A — Students & Courses
- Group B — Teaching & Grades
- Every box above = variables you can write today

THE BIG PICTURE

One Portal, two halves

The same system seen from two sides. Each group owns one half — and they share the Student and the Course.

Group A — the student's side

Register, log in, browse & enrol in courses, see your timetable, mark attendance.

Group B — the lecturer's side

Create assignments, collect submissions, grade them, give feedback, print class reports.

The shared core

Both halves revolve around the same two things: a **Student** and a **Course**. That's exactly what you modelled a minute ago.

GROUP A · STUDENT & COURSES

What Group A is building

The student experience

A student signs up, logs in, finds courses, enrols, and checks their attendance — all from the console.

What it must remember

Students, the courses on offer, who enrolled in what, and each day's attendance.

From your diagrams

Your class diagram already names these. Let's turn each class into variables you can write today.

GROUP A · YOUR CLASSES

Four things to model



Student

name, number, year, GPA



Course

code, title, credits, seats



Enrollment

which student, which course



Attendance

present today? yes / no

The pattern

Each "class" is really a **group of variables** that belong together. Today: the variables. Lecture 9: bundling them into a real class.

GROUP A · IN CODE

A Student — as variables

```
// one student, using today's types
string name   = "Sara Khan";   // text
int    number = 1024;          // whole number
int    year   = 2;             // year of study
double gpa    = 3.75;          // decimals
bool   active = true;          // enrolled now?
```

`string` name
Any text — in the full compiler.

`int` number, year
Counts — no decimals needed.

`bool` active
A yes/no flag — the start of real logic.

GROUP A · IN CODE

A Course — as variables

```
string code    = "CS101";      // course code
string title   = "Intro to C++";
int    credits = 6;            // ECTS credits
int    seats   = 30;           // places available
bool   isOpen  = true;         // can students still enrol?
```



Notice

Same five types, different meaning. That's the whole trick — pick the type that fits the data.

GROUP A · THE USER'S JOURNEY

What happens when a student enrols

Your **sequence diagram** is just this story, step by step. Each step reads or changes a variable.

1 · Log in

Check the student's `number` and password.

2 · Browse courses

Show each Course where `isOpen == true` and `seats > 0`.

3 · Enrol

Make an Enrollment, and do `seats = seats - 1`.

4 · Mark attendance

Each class: set `bool present` for the day.

GROUP B · TEACHING & GRADES

What Group B is building

The lecturer experience

A lecturer creates an assignment, students submit, the lecturer grades each one and gives feedback.

What it must remember

Assignments, every submission, the grade and feedback, and a class report at the end.

From your diagrams

Your class diagram names the Lecturer, Assignment, Submission and Grade. Same move — classes become variables.

GROUP B · YOUR CLASSES

Four things to model



Lecturer

name, staff ID



Assignment

title, max marks, due



Submission

which student, when



Grade

marks, feedback, pass?

Same pattern

A Grade is just: a `double` mark, a `string` feedback, and a `bool` passed. You can write that *now*.

GROUP B · IN CODE

An Assignment — as variables

```
string title    = "Lab 1: Hello World";
string course   = "CS101";
int    maxMarks = 100;          // out of 100
int    weight   = 10;           // % of final grade
bool   isOpen   = true;         // still accepting work?
```



Tie-in

`course = "CS101"` is the **same** code Group A used. That's where the two halves meet.

GROUP B · IN CODE

A Grade — as variables

```
int    studentNo = 1024;       // whose grade
double marks     = 82.5;       // the score (decimals!)
string feedback  = "Great work, tidy code.";
bool   passed    = marks >= 50;  // automatic pass check
```

`double` marks
82.5 — the `/2.0` lesson, for real.

`bool` passed
A comparison fills it in — no typing yes/no.

`string` feedback
Text back to the student.

GROUP B · THE USER'S JOURNEY

What happens when work is graded

Your sequence diagram, in four steps — each one touches a variable.

1 · Create assignment

Set `title`, `maxMarks`, `isOpen = true`.

2 · Student submits

Record a Submission with the student's `number`.

3 · Grade it

Set `marks`, then `passed = marks >= 50`.

4 · Class report

Average all `marks` — a `double`, of course.

BOTH GROUPS

Where your two projects meet



Group A owns the **Student**





Group B gives it a **Grade**

The shared key

A student's `number` and a course `code` link the two halves. If both groups use the same names, your Portals snap together.

THE ROAD AHEAD

From loose variables to a real class

Today each thing is separate variables. Soon you'll bundle them so one `Student` carries all its data at once:

```
struct Student {
    string name;
    int    number;
    double gpa;
    bool   active;
};  // Lecture 9 — don't worry yet!
```

Lecture 9
Structs & objects — bundle the variables.

Lecture 7
Vectors — a *list* of students, not just one.

Lecture 10
Files — save the Portal so it remembers.

THIS WEEK · BOTH GROUPS

Your task before we build more

1 · List your variables

For each class in your diagram, write down every piece of data and pick its C++ type (int / double / char / bool / string).

2 · Code one object

Pick your main class (Student for A, Assignment for B) and store one real example in variables — like we just did.

3 · Agree on shared names

Both groups: settle on `number` for a student and `code` for a course, so your halves connect.

4 · Save it

Keep the file — it's the first real brick of your Portal.

LET'S TALK OPEN FLOOR

Project discussion

Bring your diagrams up. Let's go through these together:

Which classes did you choose?
Walk me through your class diagram.

Where do A and B connect?
Agree the shared names right now.

What's the hardest piece of data?
Anything that doesn't fit a simple type?

What will you demo next week?
One small, runnable piece each.

RECAP

What you learned today

Concepts

Variables are named boxes. Each type (int, double, char, bool, string) fits a different kind of data.

Skills

You declared variables, read input with `cin`, avoided the int-division trap, and modelled a real Student record.

Milestone

Your programs can now remember and use data — the foundation of the whole Portal.

KEEP THESE WORDS

Today's vocabulary — click any word for a reminder

`int` Whole number	`double` Decimal	`char` One letter	`bool` true / false
`string` Text	`cin` Read input	`const` Never changes	`=` Assign a value

BEFORE NEXT TIME

Practice (15–20 min)

Model a Course

Store course code, title, credits and semester in the right types, and print a course card.

Compute an average

Three test marks → print the average as a `double`. Watch the `/ 3.0`!

Save it

Add these to your code folder — they're pieces of the project.

NEXT LECTURE

Lecture 3 — Operators & Expressions

Today you stored data. Next time you'll **compute** with it — the maths behind grades, averages and fees.



- Arithmetic: + − × ÷ and remainder
- Comparisons: is this mark a pass?
- Build a grade calculator for the Portal



You built your first record.

Your programs can remember data now — that's a real step toward the Portal. See you next lecture.

Dr. Mir Hassan · Associate Professor in AI & Trustworthy Digital Systems